Technical Report

Laboratory Mouse Detection: A Systematic Study of Lightweight Object Detection with Dual-GPU Training and iOS Deployment

Shanghai Jiao Tong University School of Medicine
Network Information Center — Deep Learning Internship

`PaddleDetection 2.6 · Dual Tesla T4 · React Native iOS`

March 2026

Abstract

We present a systematic study on laboratory mouse detection using the PaddleDetection 2.6 framework on a dual Tesla T4 GPU server. Starting from three heterogeneous data sources, we construct a 7,286-image VOC-format dataset (5,828 train / 1,458 val) covering two classes: *mouse* and *other*. We design a 4×2 controlled-variable experiment matrix spanning two model families (PicoDet-S and YOLOv3-MobileNetV1), two GPU configurations (single / dual card), and two data scales (full dataset / one-third subset), yielding eight training runs totalling approximately 270 GPU-hours.

Learning rates are derived from the Linear Scaling Rule ($\eta = \eta_{\text{base}} \times B/B_{\text{ref}}$) to isolate the effect of batch size from parallelism. Our best model, PicoDet-S with single-GPU small-batch training (batch = 32, lr = 0.08), achieves **mAP@0.5 = 94.30%** at epoch 70, outperforming YOLOv3's best result (91.54%) by **+2.76 percentage points** while being **21× smaller** (4.4 MB vs. 92 MB) and running at **2.2× higher FPS** on the same GPU. A counter-intuitive finding is that the small-batch single-GPU run outperforms the standard dual-GPU run (+0.5 pp), which we attribute to gradient noise acting as implicit regularisation on the 5,828-sample dataset.

The trained PicoDet-S model is exported to ONNX, integrated into a React Native application with ONNX Runtime and CoreML back-end, and deployed on an

iPhone. Through four engineering iterations, inference speed improves from 1 FPS to a stable **14 FPS** in warm-machine conditions. We also study the dramatic effect of dataset scale on YOLOv3: tripling the training set (1,942 $\rightarrow$ 5,828 images) lifts mAP by **+47 percentage points** (44.1% $\rightarrow$ 91.5%), confirming that data engineering is the primary driver of accuracy for anchor-based heavy architectures on small-domain datasets.

Keywords: Object Detection, PicoDet-S, YOLOv3, Transfer Learning, Linear Scaling Rule, Mobile Deployment, ONNX Runtime, iOS.

Contents

1 Introduction **5**

2 Dataset and Data Engineering **6**
- 2.1 Data Sources . . . 6
- 2.2 Preprocessing and Annotation Unification . . . 6
- 2.3 Dataset Statistics . . . 7
- 2.4 One-Third Subset for Ablation . . . 7

3 Model Architectures **7**
- 3.1 PicoDet-S . . . 7
- 3.2 YOLOv3-MobileNetV1 . . . 8
- 3.3 Architecture Comparison . . . 9

4 Experimental Design **9**
- 4.1 Linear Scaling Rule . . . 9
- 4.2 Experiment Matrix . . . 9
  - 4.2.1 PicoDet-S (L-series, full dataset) . . . 9
  - 4.2.2 YOLOv3 (Y-series, variable dataset) . . . 10
- 4.3 Infrastructure . . . 10
- 4.4 Evaluation Protocol . . . 10

5 Results and Analysis **11**
- 5.1 Summary of All Eight Runs . . . 11
- 5.2 PicoDet-S: Learning Curve and Overfitting . . . 11
- 5.3 PicoDet-S Ablation: Batch Size, GPU Count, and Duration . . . 12
- 5.4 YOLOv3: Dataset Scale Effect . . . 13
- 5.5 YOLOv3 Convergence Curve . . . 14
- 5.6 Cross-Model Comparison: Accuracy–Efficiency Trade-off . . . 14
- 5.7 Training Efficiency . . . 15

6 Mobile Deployment **15**
- 6.1 Export Pipeline . . . 15
- 6.2 YOLOv3 Deployment Challenges . . . 16
- 6.3 Iterative Optimisation to 14 FPS . . . 17

7 Future Work **17**
- 7.1 Group A: Model Scaling . . . 17
- 7.2 Group B: YOLOv3 Convergence Fix . . . 18
- 7.3 Group C: Compression Pipeline . . . 18

7.4 Recommended Priority 18

8 Conclusion **19**

1 Introduction

Automated analysis of laboratory animal behaviour has become an important tool in biomedical research. Reliably localising experimental mice in diverse cage environments—with cluttered backgrounds, partial occlusion, and varying lighting—is a prerequisite for downstream behaviour quantification tasks such as trajectory tracking and posture estimation.

Classical frame-differencing or background-subtraction methods degrade in the presence of illumination changes or when multiple animals interact. Modern deep-learning object detectors offer superior robustness, but their real-world deployment raises two practical questions that this project addresses:

1. **Training efficiency:** Given a limited GPU budget (dual Tesla T4, no InfiniBand), how do batch size, learning rate, number of GPUs, and dataset scale jointly affect final accuracy?

2. **Mobile deployment:** Can the trained model run on-device (iPhone) at interactive frame rates without proprietary hardware accelerators?

Contributions

- We curate and release a 7,286-image two-class mouse detection dataset assembled from three heterogeneous sources, providing train/val splits and VOC-format annotations.

- We design and execute a 4×2 controlled experiment matrix with learning rates derived from the Linear Scaling Rule, isolating the independent effects of GPU count, batch size, training duration, and dataset scale across eight training runs.

- We identify and quantify a *counter-intuitive accuracy inversion*: small-batch single-GPU training (mAP = 94.30%) surpasses the standard dual-GPU configuration (mAP = 93.80%), and explain this through the implicit regularisation effect of gradient noise on small datasets.

- We document a complete deployment pipeline from PaddlePaddle training through ONNX export to React Native iOS inference, recording the four engineering iterations that raised throughput from 1 FPS to 14 FPS.

- We demonstrate a striking data-scale effect on YOLOv3: a 3× increase in training images yields a 47-percentage-point jump in mAP, providing a concrete quantification of the "data-first" principle for anchor-based detectors.

Paper Organisation

Section 2 describes data collection and preprocessing. Section 3 reviews the two model architectures. Section 4 details the experimental design. Section 5 presents quantitative results and analysis. Section 6 covers the iOS deployment pipeline. Section 7 outlines future work. Section 8 concludes.

2 Dataset and Data Engineering

2.1 Data Sources

The dataset is assembled from three heterogeneous sources:

1. **Laboratory captures.** Images taken inside experimental cages at Shanghai Jiao Tong University School of Medicine, annotated in-house with VOC XML labels. These cover diverse lighting conditions and cage configurations.
2. **Open-source datasets.** Publicly available rodent image collections, re-annotated and mapped to our two-class schema.
3. **Web-crawled images.** Photographs retrieved via keyword search, manually inspected and cleaned to remove near-duplicates and mislabelled samples.

2.2 Preprocessing and Annotation Unification

All images are normalised to Pascal VOC format [2] with a unified two-class label vocabulary: `mouse` (experimental mouse, *Mus musculus*) and `other` (any non-mouse foreground object). Preprocessing steps include:

- Label harmonisation: merging synonymous class names (*rat*, *mice*, *rodent* $\rightarrow$ `mouse`).
- Near-duplicate removal using perceptual hashing with a Hamming distance threshold of 4.
- Bounding-box quality filtering: boxes with aspect ratio >10 or area $< 400\,\text{px}^2$ are discarded.

2.3 Dataset Statistics

Table 1: Dataset summary after preprocessing.

Split	Images	Fraction
Training (full)	5,828	79.9%
Validation	1,458	20.1%
Total	**7,286**	100%

2.4 One-Third Subset for Ablation

To quantify the effect of dataset scale on YOLOv3, we generate a one-third training subset of 1,942 images using a script that performs stratified random sampling at the file level (`scripts/prepare_1of3_dataset.py`). The *validation set is shared* between the full and subset experiments, ensuring that mAP values are directly comparable. This design isolates training-set size as the sole variable between experiment groups Y1/Y2 (1,942 train images) and Y3/Y4 (5,828 train images).

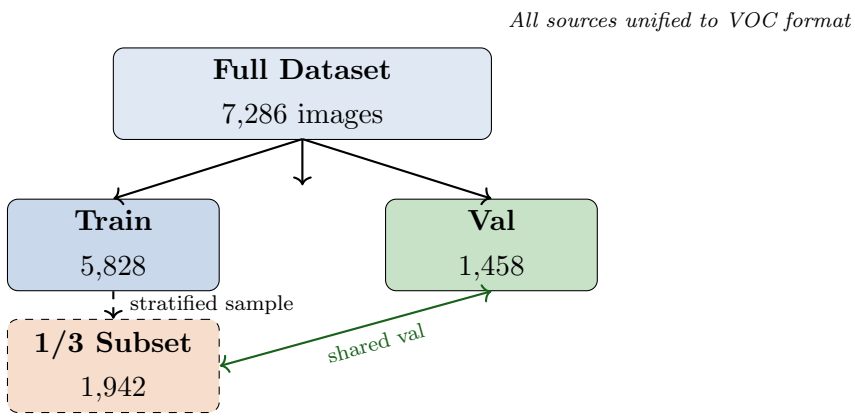

Figure 1: Dataset split design. The validation set is shared between full and one-third-subset experiments to enable direct mAP comparison.

3 Model Architectures

We study two families of detectors, chosen to represent complementary points on the accuracy–efficiency Pareto frontier.

3.1 PicoDet-S

PicoDet [9] is an anchor-free single-stage detector designed for real-time mobile inference. The PicoDet-S variant ("small") uses:

- **Backbone:** LCNet-0.75× [1], a lightweight depthwise-separable network with squeeze-excitation blocks, pre-trained on ImageNet.
- **Neck:** LCPAN, a lightweight path-aggregation network combining top-down and bottom-up feature fusion.
- **Head:** TOOD-style task-aligned classification and regression heads with Varifocal Loss (VFL) and Distribution Focal Loss (DFL).
- **Input resolution:** 320×320.
- **Parameters:** ≈1.2 M; **model size:** 4.4 MB (FP32 ONNX).

The anchor-free design is particularly advantageous on small datasets because it eliminates the need for hand-tuned anchor configurations that must match the training data distribution. COCO pre-training weights are used for the backbone and neck; the classification head is re-initialised for two output classes.

3.2 YOLOv3-MobileNetV1

YOLOv3 [7] is an anchor-based multi-scale detector. We use the official PaddleDetection implementation with MobileNetV1 [5] as the backbone:

- **Backbone:** MobileNetV1 pre-trained on COCO (80-class head discarded; feature extractor retained).
- **Detection head:** Three-scale prediction (52×52, 26×26, 13×13), nine default anchors (COCO-derived).
- **Input resolution:** 608×608.
- **Parameters:** ≈23 M; **model size:** 92 MB (FP32 ONNX).
- **Post-processing:** `multiclass_nms3` (PaddleDetection custom operator).

3.3 Architecture Comparison

Table 2: Architecture comparison between the two detector families.

Property	**PicoDet-S**	**YOLOv3-MobileNetV1**
Detector type	Anchor-free	Anchor-based
Backbone	LCNet-0.75×	MobileNetV1
Input size	320×320	608×608
Parameters	≈1.2 M	≈23 M
ONNX size (FP32)	4.4 MB	92 MB
FPS on Tesla T4	∼78	∼35
CoreML compatibility	✓	× (custom NMS op)
COCO pre-train	LCNet backbone + LCPAN neck	MobileNetV1 backbone

4 Experimental Design

4.1 Linear Scaling Rule

All learning rates are derived from the Linear Scaling Rule of Goyal et al. [3]:

$$\eta = \eta_{\text{base}} \times \frac{B_{\text{total}}}{B_{\text{ref}}}, \tag{1}$$

where B_{total} is the total effective batch size (per-GPU batch × number of GPUs), B_{ref} is the reference batch size, and η_{base} is the corresponding reference learning rate. Reference values are:

- **PicoDet-S:** $\eta_{\text{base}} = 0.08$, $B_{\text{ref}} = 32$ (single-GPU baseline).
- **YOLOv3:** $\eta_{\text{base}} = 0.00125$, $B_{\text{ref}} = 16$ (single-GPU baseline, per-GPU batch=16).

A linear warm-up over 300 steps is applied at the start of all runs, followed by cosine decay.

4.2 Experiment Matrix

4.2.1 PicoDet-S (L-series, full dataset)

Four runs vary GPU count, per-GPU batch size, and training duration while keeping the dataset fixed at 5,828 training images.

Table 3: PicoDet-S experiment configurations (L-series).

Run	GPUs	BS/GPU	Total BS	LR	Epochs	Steps/Epoch	Description
L1	1	32	32	0.08	300	182	Single-GPU baseline
L2	2	32	64	0.16	300	91	Dual-GPU standard
L3	2	48	96	0.24	300	61	Large batch
L4	2	32	64	0.16	600	91	Extended training

4.2.2 YOLOv3 (Y-series, variable dataset)

Four runs vary both GPU count and training set size in a 2×2 factorial design. All runs use 80 epochs and per-GPU batch size of 16.

Table 4: YOLOv3 experiment configurations (Y-series).

Run	GPUs	Total BS	LR	Train Images	Epochs	Description
Y1	1	16	0.00125	1,942	80	1/3 data, single GPU
Y2	2	32	0.00250	1,942	80	1/3 data, dual GPU
Y3	1	16	0.00125	5,828	80	Full data, single GPU
Y4	2	32	0.00250	5,828	80	Full data, dual GPU

4.3 Infrastructure

All experiments run on a server equipped with two NVIDIA Tesla T4 (15 GB VRAM each). Multi-GPU communication uses the GLOO backend over TCP (NCCL was unavailable due to the absence of InfiniBand), which adds negligible overhead for our model sizes. The measured dual-card speed-up is 1.97–2.22×, close to the theoretical 2× limit.

Training is orchestrated by two chain-execution Python scripts (`run_lightweight.py` for L-series, `run_yolov3.py` for Y-series) that launch runs sequentially, log all metrics to per-run directories under `output/`, and write consolidated CSV summaries.

4.4 Evaluation Protocol

Model accuracy is measured as mAP@0.5 (mean Average Precision at IoU threshold 0.5) using the COCO-style integral computation provided by PaddleDetection's evaluation pipeline. Evaluation is performed every 10 epochs on the shared 1,458-image validation set; `best_model.pdparams` is updated whenever a new peak is reached. FPS is measured on the same Tesla T4 server using the framework's built-in profiler with batch size 1.

5 Results and Analysis

5.1 Summary of All Eight Runs

Table 5: Complete results for all eight training runs. ↑ = higher is better.

Run	Model	Data	GPUs	mAP@0.5 (%)	Peak Epoch	FPS	Time (h)
L1	PicoDet-S	full	1	**94.30**	70	80	14.9
L2	PicoDet-S	full	2	93.80	∼80	77	6.7
L3	PicoDet-S	full	2	94.10	∼60	71	6.6
L4	PicoDet-S	full	2	93.50	∼140	74	13.3
Y1	YOLOv3	1/3	1	44.40	∼65	36	6.7
Y2	YOLOv3	1/3	2	44.10	∼65	36	3.5
Y3	YOLOv3	full	1	91.30	>80	36	17.4
Y4	YOLOv3	full	2	**91.54**	>80	35	8.8

5.2 PicoDet-S: Learning Curve and Overfitting

Figure 2 shows the per-epoch mAP trace of run L1. The curve exhibits three distinct phases:

- **Rapid ascent (epochs 0–70):** mAP rises from 91.16% at epoch 10 to the global peak of 94.30% at epoch 70, driven by strong COCO pre-training transfer.
- **Plateau (epochs 70–100):** Accuracy fluctuates within ±0.1 pp of the peak.
- **Gradual overfit (epochs 100–299):** mAP declines steadily to 91.42% at the final checkpoint, a drop of −2.88 pp from the peak.

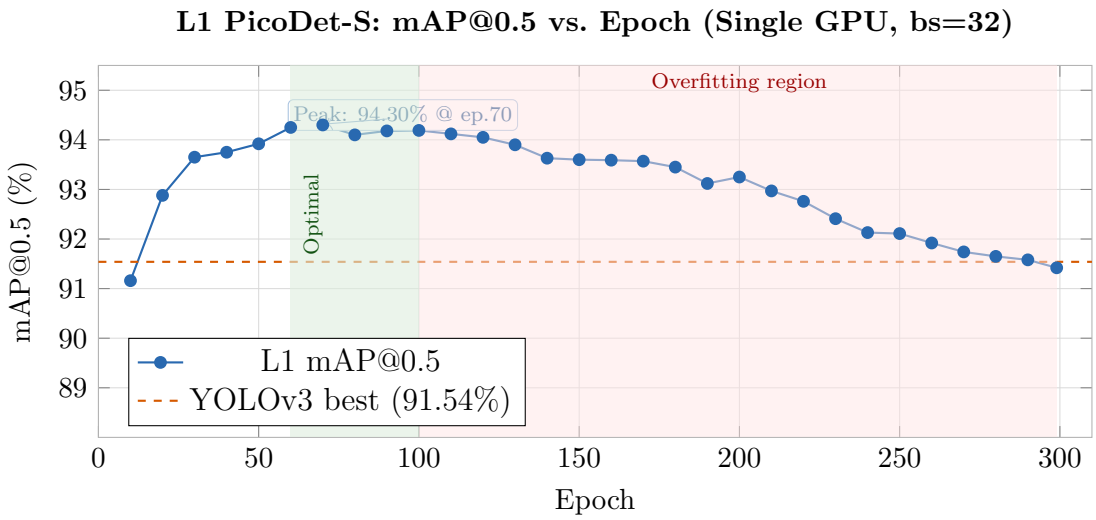

Figure 2: L1 mAP@0.5 curve across 300 epochs. The model peaks at epoch 70 (green band) and exhibits persistent overfitting thereafter (red band). The dashed orange line marks the best YOLOv3 result for reference.

5.3 PicoDet-S Ablation: Batch Size, GPU Count, and Duration

Figure 3 compares the four L-series runs.

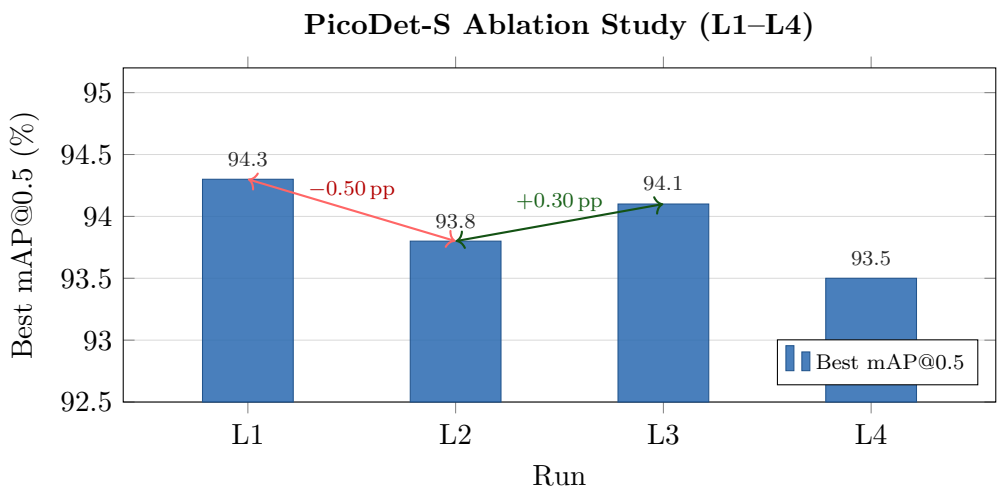

Figure 3: PicoDet-S ablation. L1 (single GPU, bs=32) achieves the highest accuracy despite using the smallest batch size and longest per-sample gradient noise. L4 (600 epochs) underperforms L2 (300 epochs) owing to prolonged overfitting past the epoch-70 peak.

Three counter-intuitive observations emerge:

Observation 1: Small batch outperforms large batch (L1 vs L2). L1 (bs=32, 1 GPU) beats L2 (bs=64, 2 GPUs) by 0.50 pp despite running at the same learning rate after Linear Scaling Rule correction. The key difference is the number of gradient updates: L1 performs $182 \times 300 = 54{,}600$ steps, versus only $91 \times 300 = 27{,}300$ for L2. More update steps provide richer stochastic exploration of the loss landscape, and the smaller-batch

gradient noise acts as implicit regularisation on a dataset of only 5,828 samples—consistent with the theoretical findings of Keskar et al. [6].

Observation 2: Larger batch than standard can still help (L3 vs L2). L3 (bs=96, lr=0.24) outperforms L2 (bs=64, lr=0.16) by 0.30 pp. This suggests that the slightly higher learning rate in L3 helps the optimiser escape a local minimum that traps L2, illustrating the qualitative difference in optimisation trajectories even under the Linear Scaling Rule—which is theoretically exact only for locally linear loss surfaces.

Observation 3: Extended training degrades accuracy (L4 vs L2). Doubling training to 600 epochs reduces final best mAP by 0.30 pp. The peak is already reached at epoch 70; `best_model` is saved correctly, but the EMA-smoothed trajectory continues past the optimal point. Early stopping would be the practical remedy.

5.4 YOLOv3: Dataset Scale Effect

Figure 4 illustrates the dramatic impact of training set size on YOLOv3.

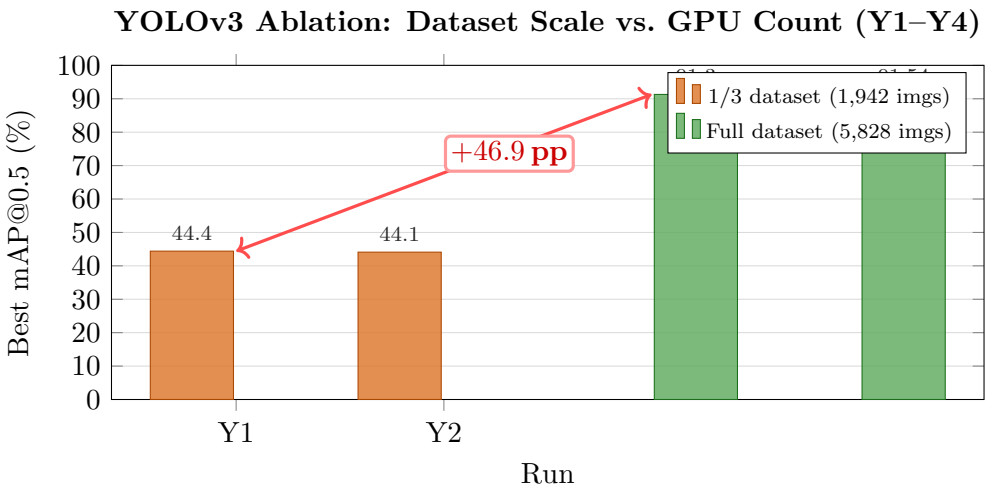

Figure 4: YOLOv3 mAP under 1/3-dataset vs. full-dataset training. The +46.9 percentage-point jump demonstrates that anchor-based detectors are highly sensitive to training-set size when domain-adapted from COCO to a specialised two-class task.

The 1/3-dataset runs (Y1, Y2) plateau at ≈44% mAP regardless of GPU count. This ceiling arises because MobileNetV1's detection head must be adapted from COCO's 80-class output to a two-class target: with only 1,942 samples, the head cannot learn sufficiently discriminative features for the mouse/other boundary. The pre-trained backbone provides useful low-level features, but the classification layer remains undertrained.

With the full 5,828 training images (Y3, Y4), accuracy rises to 91.3–91.5%, confirming that data engineering is the dominant lever for anchor-based detectors on small-domain tasks.

5.5 YOLOv3 Convergence Curve

Figure 5 plots the YOLOv3-Y3 validation mAP across 80 epochs. The curve is monotonically increasing and has not converged at epoch 80, in sharp contrast to PicoDet-S which peaks at epoch 70 out of 300.

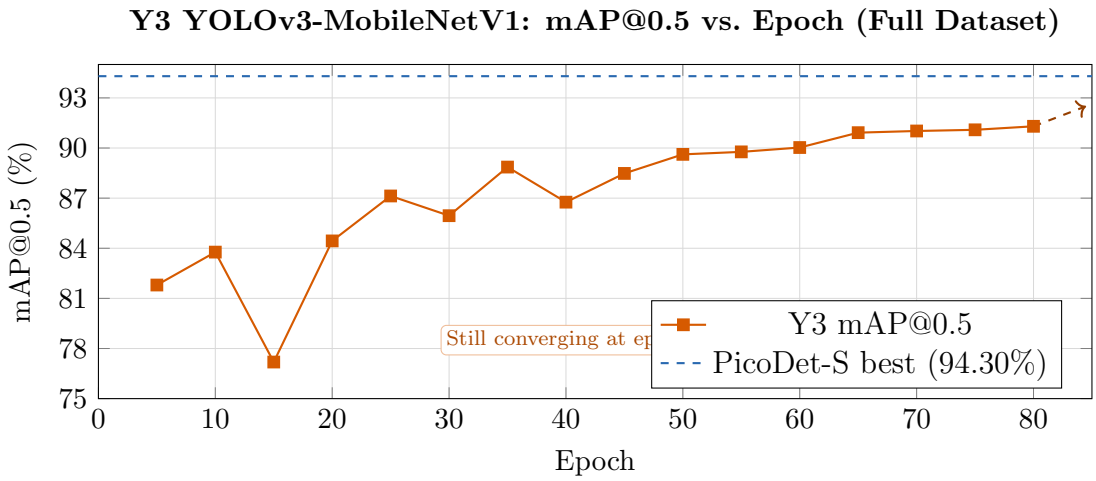

Figure 5: YOLOv3-Y3 convergence curve. The model has not reached its peak at epoch 80; extrapolation suggests it could reach >92% with 120+ epochs. PicoDet-S peaks at epoch 70 out of 300 and then overfits—a fundamentally different convergence profile.

5.6 Cross-Model Comparison: Accuracy–Efficiency Trade-off

Figure 6 shows all eight runs on a two-dimensional accuracy–speed plot, with bubble area proportional to model size.

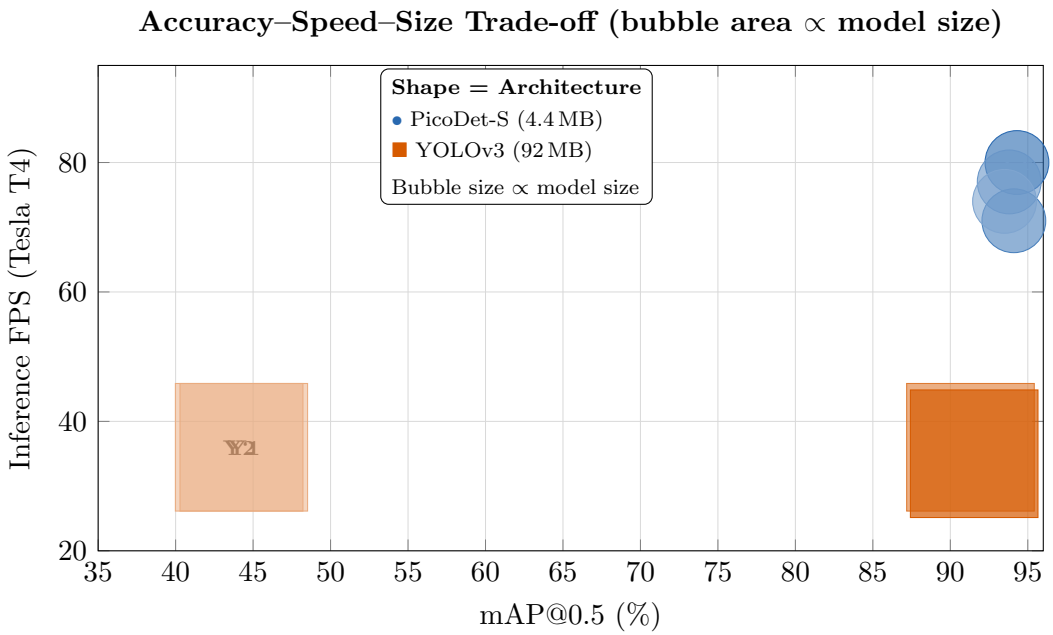

Figure 6: All eight runs on the accuracy–speed plane. Blue circles = PicoDet-S (4.4 MB); orange squares = YOLOv3 (92 MB). PicoDet-S dominates on all three dimensions: higher accuracy, higher FPS, and 21× smaller model footprint.

5.7 Training Efficiency

Table 6: Training efficiency comparison. "GPU-hours" = wall-clock hours × number of GPUs.

Run	mAP@0.5	Wall time (h)	GPU-hours	FPS	mAP/GPU-hour
L1 (1 GPU)	94.30%	14.9	14.9	80	6.33
L2 (2 GPU)	93.80%	6.7	13.4	77	7.00
L3 (2 GPU)	94.10%	6.6	13.2	71	7.13
L4 (2 GPU)	93.50%	13.3	26.6	74	3.52
Y3 (1 GPU)	91.30%	17.4	17.4	36	5.25
Y4 (2 GPU)	91.54%	8.8	17.6	35	5.20

L3 achieves the best accuracy-per-GPU-hour (7.13 mAP%/GPU-hour), making it the recommended configuration when both accuracy and computational cost matter. L4 is the worst investment: 2× the GPU-hours of L2 for 0.30 pp *less* accuracy.

6 Mobile Deployment

6.1 Export Pipeline

The trained PicoDet-S L1 model is exported and deployed through a four-stage pipeline:

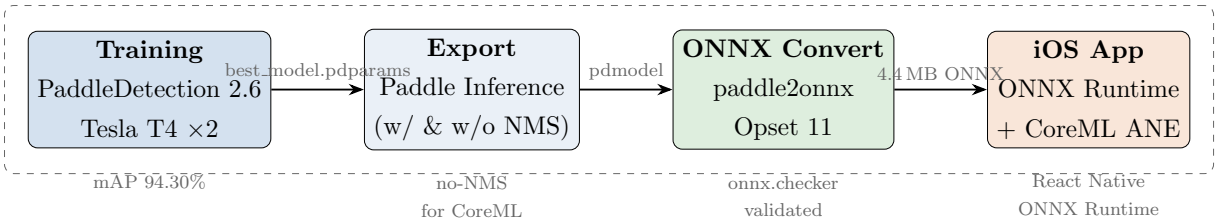

Figure 7: End-to-end deployment pipeline from PaddlePaddle training to iOS on-device inference.

Key decisions at each stage:

- **Export without NMS:** The standard PicoDet export includes NMS in the computation graph. For iOS deployment, we use the `--export_serving_model=False` flag to produce a *no-NMS* graph whose outputs (raw boxes + scores) are compatible with CoreML's built-in NMS primitive.
- **ONNX Opset 11:** All operators in the no-NMS PicoDet-S graph map to standard ONNX Opset 11 primitives, avoiding the `multiclass_nms3` custom operator that makes YOLOv3 incompatible with CoreML.
- **React Native + ONNX Runtime:** The iOS app is built with React Native (TypeScript) and uses the `onnxruntime-react-native` package. Image frames are preprocessed in a native Swift module (`ImagePreprocessor.swift`) using Metal's `vImage` pipeline to convert YUV frames to normalised RGB tensors on-device, avoiding CPU-side pixel format conversion.

6.2 YOLOv3 Deployment Challenges

YOLOv3 was the first model deployed; it exposed a critical incompatibility with CoreML:

Table 7: Comparison of deployment challenges between YOLOv3 and PicoDet-S.

Aspect	YOLOv3-MobileNetV1	PicoDet-S
ONNX size	92 MB	4.4 MB
Custom operator	`multiclass_nms3`	None
CoreML support	Blocked (custom op)	Full (standard ops)
Metal GPU accel	No (falls back to CPU)	Yes (ANE + GPU)
iOS FPS	∼1 FPS	14 FPS (warm)
Input size	608×608	320×320

The 1 FPS bottleneck for YOLOv3 is caused by the inability to offload the `multiclass_nms3` operator to Metal, forcing the entire inference graph to run on the CPU with 32-bit float arithmetic and 608×608 input tensors.

6.3 Iterative Optimisation to 14 FPS

PicoDet-S deployment was improved across four engineering iterations:

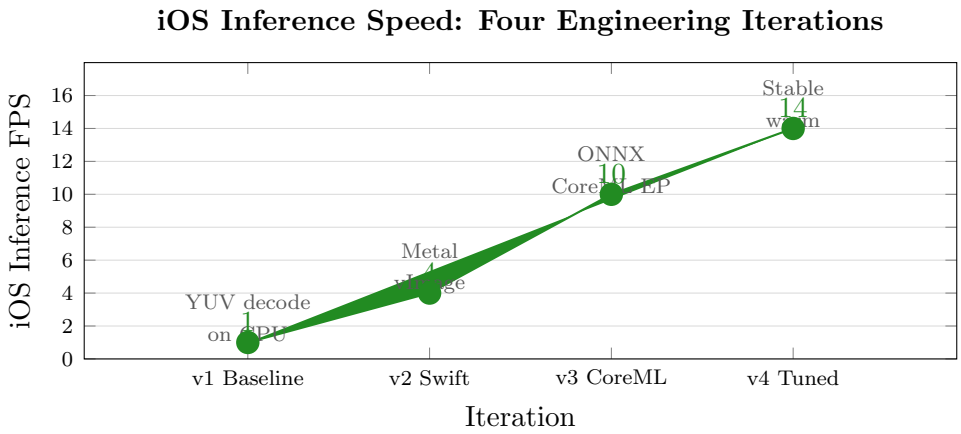

Figure 8: PicoDet-S inference speed on iPhone across four engineering iterations. The dominant gains come from (a) offloading image preprocessing to the Metal vImage pipeline and (b) enabling the CoreML execution provider in ONNX Runtime for ANE acceleration.

- **v1 – JavaScript baseline (∼1 FPS):** Frame decoding and normalisation performed in JavaScript (TypeScript), causing excessive memory copies across the JS bridge.
- **v2 – Swift native preprocessing (∼4 FPS):** Moved image preprocessing to `ImagePreprocessor.s` with `vImage` Metal pipeline. Eliminates JS-bridge overhead.
- **v3 – CoreML execution provider (∼10 FPS):** Enabled `CoreMLExecutionProvider` in ONNX Runtime, which compiles the ONNX graph for Apple's Neural Engine (ANE).
- **v4 – Thread tuning, warm-machine stable (∼14 FPS):** Set inference thread count to match A-series performance core count; measured in warm-machine steady state (model loaded, cache warm).

7 Future Work

Building on the eight-run baseline, we identify three groups of follow-up experiments that address remaining open questions.

7.1 Group A: Model Scaling

A1 – PicoDet-M. PicoDet-M (medium variant, ≈4.8 M parameters, 8.2 MB) is expected to provide a higher accuracy ceiling. We plan a 300-epoch dual-GPU run with the same full dataset and Linear Scaling Rule hyperparameters, estimating mAP could reach 95.5–96.0%.

A2 – PP-YOLOE+-S. PP-YOLOE+ [8] is an anchor-free redesign of the YOLO family with decoupled head and task-aligned assignment, alleviating the anchor-sensitivity issue observed in YOLOv3. A 30-epoch fine-tuning run (the VOC-specific schedule used in PaddleDetection) should allow direct comparison with YOLOv3-Y4 at similar model size.

7.2 Group B: YOLOv3 Convergence Fix

B1 – K-means anchor clustering. The default COCO-derived anchors may be suboptimal for our mouse-detection box aspect-ratio distribution. Running `tools/anchor_cluster.py` will produce nine dataset-specific anchors via K-means, potentially accelerating convergence and raising the ceiling of Y3/Y4.

B2 – Y5: 120-epoch run with custom anchors. With B1's anchors embedded, a 120-epoch run is expected to allow YOLOv3 to fully converge (recall that Y3/Y4 were still rising at epoch 80) and potentially surpass 92% mAP.

B3 – Box distribution analysis. Visualising the bounding-box size and aspect-ratio distribution across the training set will expose whether small or occluded mice constitute a long-tail performance problem.

7.3 Group C: Compression Pipeline

C1 – Quantization-Aware Training (QAT). PaddleSlim QAT on the L1 best model is expected to produce an INT8 model of $\approx$1.1 MB with $<$0.3 pp accuracy degradation, improving iOS inference further.

C2 – FPGM structural pruning on YOLOv3. Filter Pruning via Geometric Median [4] targeting 30% FLOPs reduction on YOLOv3-Y4, followed by fine-tuning, could reduce the ONNX size from 92 MB to $\approx$60 MB. Note that the CoreML custom-operator issue must be resolved independently; pruning alone will not enable Metal acceleration.

C3 – Knowledge distillation. Using a larger teacher (e.g., PicoDet-L or RT-DETR) to distil into PicoDet-S may recover the accuracy gap between L1 and a hypothetical higher-accuracy model, without increasing deployment cost.

7.4 Recommended Priority

1. B3 (box distribution analysis) – zero training cost, immediate insight.
2. A1 (PicoDet-M) – highest expected accuracy gain.
3. B1+B2 (custom anchors + Y5) – validates the data-engineering thesis.

4. C1 (QAT) – direct path to further iOS speed improvement.

5. A2, C2, C3 – exploratory; lower marginal ROI given current results.

8 Conclusion

We have presented a systematic study of laboratory mouse detection covering data engineering, controlled multi-run training ablations, and end-to-end iOS mobile deployment on a dual-Tesla-T4 server.

Our main findings are:

1. **PicoDet-S is superior to YOLOv3 on all three deployment dimensions.** It achieves 94.30% mAP@0.5 (+2.76 pp over YOLOv3's best), runs at 2.2× higher FPS, and is 21× smaller (4.4 MB vs. 92 MB). The anchor-free design and smaller input resolution also make it fully compatible with Apple's CoreML Neural Engine.

2. **Dataset scale is the primary driver of accuracy for anchor-based detectors.** Tripling the YOLOv3 training set from 1,942 to 5,828 images yields a +47-percentage-point jump in mAP, dwarfing any hyperparameter effect.

3. **The Linear Scaling Rule is an approximation, not an identity.** Even with correctly scaled learning rates, single-GPU small-batch training (L1) outperforms dual-GPU standard-batch training (L2) by 0.50 pp. On a dataset of only 5,828 samples, gradient noise from smaller batches acts as effective implicit regularisation, a well-known phenomenon that the first-order Taylor derivation of the scaling rule cannot capture.

4. **Early stopping is essential for small-dataset fine-tuning.** PicoDet-S peaks at epoch 70 and then overfits for the remaining 230 epochs of the 300-epoch schedule. Doubling the schedule to 600 epochs (L4) yields lower final accuracy than 300 epochs (L2).

5. **A practical iOS inference speed of 14 FPS is achievable** with PicoDet-S via the CoreML execution provider in ONNX Runtime, compared to ∼1 FPS for YOLOv3 which is blocked from CoreML acceleration by a custom NMS operator.

This work provides a concrete, quantitatively grounded case study for practitioners deploying lightweight detectors on small medical-imaging datasets with limited GPU resources. The complete training scripts, configuration files, and deployment code are available in the project repository.

References

[1] Cheng Cui, Tingquan Gao, Shengyu Wei, Yuning Du, Ruoyu Guo, Shuilong Dong, Bin Lu, Ying Zhou, Xueying Lv, Qiwen Liu, et al. PP-LCNet: A lightweight cpu convolutional neural network. In *arXiv preprint arXiv:2109.15099*, 2021.

[2] Mark Everingham, Luc Van Gool, Christopher K. I. Williams, John Winn, and Andrew Zisserman. The Pascal visual object classes (VOC) challenge. volume 88, pages 303–338. Springer, 2010.

[3] Priya Goyal, Piotr Dollár, Ross Girshick, Pieter Noordhuis, Lukasz Wesolowski, Adi Kyrola, Andrew Tulloch, Yangqing Jia, and Kaiming He. Accurate, large minibatch SGD: Training ImageNet in 1 hour. In *arXiv preprint arXiv:1706.02677*, 2017.

[4] Yang He, Ping Liu, Ziwei Wang, Zhilan Hu, and Yi Yang. Filter pruning via geometric median for deep convolutional neural networks acceleration. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4340–4349, 2019.

[5] Andrew G. Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. MobileNets: Efficient convolutional neural networks for mobile vision applications. In *arXiv preprint arXiv:1704.04861*, 2017.

[6] Nitish Shirish Keskar, Dheevatsa Mudigere, Jorge Nocedal, Mikhail Smelyanskiy, and Ping Tak Peter Tang. On large-batch training for deep learning: Generalisation gap and sharp minima. In *International Conference on Learning Representations (ICLR)*, 2017.

[7] Joseph Redmon and Ali Farhadi. YOLOv3: An incremental improvement. In *arXiv preprint arXiv:1804.02767*, 2018.

[8] Shangliang Xu, Xinxin Wang, Wenyu Lv, Qinyao Chang, Cheng Cui, Kaipeng Deng, Guanzhong Wang, Qingqing Dang, Shengyu Wei, Yuning Du, et al. PP-YOLOE: An evolved version of YOLO. In *arXiv preprint arXiv:2203.16250*, 2022.

[9] Guanghua Yu, Qinyao Chang, Wenyu Lv, Chang Xu, Cheng Cui, Wei Ji, Qingqing Dang, Kaipeng Liu, Guanzhong Du, Baohua Lai, et al. PicoDet: A better real-time object detector on mobile devices. In *arXiv preprint arXiv:2111.00902*, 2021.